

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Case Study

Course Code:	ITA0517	Course Title:	Computer Vision
CO Assessed:	CO3 – Precision (accurate feature extraction & analysis) (BL3)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BL4)		
Student Name:	Yoga Madha A	Reg. No.:	192425058
Section / Batch:	B	Date:	12-08-2026

Code of Conduct

I Yoga Madha A-192425058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

1. Edge Detection in Low Contrast Images

An image has low contrast, causing edge detection algorithms to fail.

Analyze the problem and explain how preprocessing enhances edge detection results.

Answer:

Introduction

Edge detection is an important image-processing technique used to identify the **boundaries of objects, regions, and structures** in an image. Algorithms such as **Sobel, Prewitt, Roberts, and Canny** detect edges mainly by finding sudden changes in pixel intensity.

When an image has **low contrast**, the intensity difference between neighboring regions is very small. As a result, the gradient at the object boundary becomes weak, and the edge detection algorithm may fail to identify important edges. Therefore, suitable **preprocessing** is required before applying an edge detector.

1. Understanding the Problem

A low-contrast image contains pixels whose intensity values are concentrated within a narrow range.

For example, consider an object and its background:

Background: 120

Object: 125

The intensity difference is only:

$$125 - 120 = 5$$

This is a very small change. An edge detector may interpret this weak change as insignificant, particularly when noise is present.

In a high-contrast image, the same boundary might have:

Background: 50

Object: 200

Here:

$$200 - 50 = 150$$

The intensity difference is much larger, producing a strong gradient and a clearly detectable edge.

Basic problem

Low contrast → Small intensity difference → Weak gradient → Missing or broken edges

2. Why Edge Detection Fails

Most edge detection algorithms depend on the **rate of change of intensity**.

For an image $I(x,y)$, the gradient can be represented as:

$$\nabla I = \left[\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right]$$

The gradient magnitude is approximately:

$$|\nabla I| = \sqrt{G_x^2 + G_y^2}$$

where (G_x) and (G_y) represent intensity changes in the horizontal and vertical directions.

When the image has low contrast:

- (G_x) becomes small.
- (G_y) becomes small.
- Gradient magnitude becomes small.

- Edge pixels may not reach the detector's threshold.
- Important boundaries can disappear.

Therefore, the problem is not necessarily that the edge detection algorithm is incorrect. **The input image does not provide sufficiently strong intensity information for the algorithm to identify the edges reliably.**

3. Key Issues in the Low-Contrast Image

Several problems can occur:

Weak object boundaries

Objects may have almost the same intensity as their surrounding background. Their boundaries therefore become difficult to distinguish.

Missing edges

Weak gradients may be below the threshold used by the edge detector, causing genuine edges to be ignored.

Broken edges

An edge may be detected in some locations but not others. This produces discontinuous or broken boundaries.

Poor segmentation

If edges are required for subsequent segmentation, weak edge detection can make object extraction inaccurate.

Noise interference

In a low-contrast image, noise may have an intensity variation comparable to the actual object boundary. After contrast enhancement, this noise can become even more noticeable.

For example:

Actual boundary change: 5 intensity levels

Noise variation: 3–4 intensity levels

The algorithm may have difficulty distinguishing the actual edge from noise.

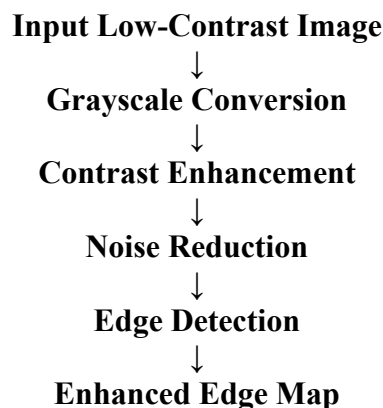
4. Role of Preprocessing

Preprocessing improves the image **before edge detection**.

The main objectives are:

1. Increase contrast.
2. Make weak boundaries more visible.
3. Reduce noise.
4. Improve the gradient between neighboring regions.
5. Provide a better-quality input to the edge detector.

A suitable processing pipeline is:



5. Contrast Enhancement

The first major preprocessing step is **contrast enhancement**.

Contrast enhancement expands or redistributes the intensity values so that differences between regions become more visible.

Two important techniques are:

- Histogram Equalization
- CLAHE

5.1 Histogram Equalization

Histogram equalization redistributes pixel intensities over a wider range.

Suppose the original image contains mostly these intensity values:

100, 105, 110, 115, 120

The range is only:

$$120 - 100 = 20$$

After enhancement, the intensities may be distributed more widely:

20, 60, 100, 150, 200

Now the intensity differences are much more visible.

Effect on edge detection

Before enhancement:

Weak intensity transition → Weak gradient → Difficult edge detection

After enhancement:

Stronger intensity transition → Stronger gradient → Better edge detection

Histogram equalization is particularly useful when the overall image has poor contrast.

6. CLAHE for Local Contrast Enhancement

Contrast Limited Adaptive Histogram Equalization (CLAHE) is often a better choice when contrast varies in different parts of an image.

Unlike ordinary histogram equalization, CLAHE divides the image into small regions called **tiles** and enhances the contrast locally.

It also limits excessive contrast amplification using a **contrast limit**.

Why CLAHE is useful

Suppose an image contains:

- A bright region
- A dark region
- A poorly contrasted object inside the dark region

Global histogram equalization may not enhance the object sufficiently. CLAHE can improve the local contrast around that object.

Therefore:

CLAHE → Local contrast improvement → Stronger local boundaries → Better edge detection

However, CLAHE should not be applied too aggressively because excessive enhancement can amplify noise and produce false edges.

7. Noise Reduction

After contrast enhancement, noise may become more visible. Therefore, noise reduction is an important preprocessing step.

A **Gaussian filter** is commonly used before edge detection.

It smooths small intensity variations while preserving larger structures.

The general processing is:

Contrast Enhancement → Gaussian Smoothing → Edge Detection

The purpose is not to remove all variations. Instead, the goal is to remove small unwanted variations while preserving meaningful boundaries.

Why smoothing is important

Suppose an image contains:

- Actual object boundary
- Random pixel fluctuations

Without smoothing, both may produce gradients.

After appropriate smoothing:

Random variations ↓

while

Major object boundary remains

This reduces false edge detection.

8. Applying Edge Detection After Preprocessing

Once contrast has been improved and noise has been reduced, an edge detector can be applied.

Sobel Operator

The Sobel operator calculates intensity changes in horizontal and vertical directions.

It uses kernels such as:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

and

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

The resulting gradient indicates where strong intensity changes occur.

After contrast enhancement, the intensity change at a weak boundary becomes stronger, so the Sobel operator can detect it more effectively.

9. Canny Edge Detection

Canny edge detection is particularly suitable when a clean and reliable edge map is required.

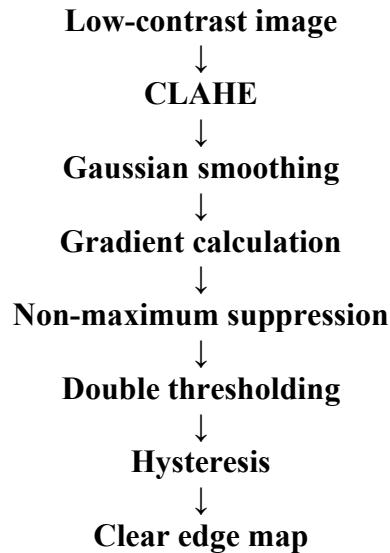
Its major stages are:

1. Noise reduction using Gaussian smoothing.
2. Gradient calculation.
3. Non-maximum suppression.
4. Double thresholding.
5. Edge tracking by hysteresis.

Low contrast can cause the gradient values to fall below the selected thresholds.

Preprocessing increases the strength of useful gradients, making it more likely that important edges are classified correctly.

Canny processing



10. Example of the Improvement

Consider an object boundary with the following pixel intensities:

Before preprocessing

120 121 122 | 123 124 125

The change across the boundary is very small.

The edge detector may produce:

No clear edge / weak edge

After contrast enhancement:

40 60 80 | 140 170 190

Now the intensity changes are much larger.

The edge detector can produce:

Strong continuous edge

Therefore, preprocessing does not directly "create" the physical boundary. Instead, it **makes the existing intensity difference sufficiently distinguishable for the edge detector.**

11. Comparison Before and After Preprocessing

Aspect	Before Preprocessing	After Preprocessing
Image contrast	Low	Improved
Intensity differences	Weak	Stronger
Gradient magnitude	Small	Larger
Object boundaries	Difficult to see	More visible
Noise influence	Relatively high	Reduced with smoothing
Edge continuity	Broken/missing	More continuous
False edges	Possible	Reduced with proper filtering
Edge detection	Poor	Improved

12. Recommended Solution

For a low-contrast image, a practical solution is:

Step 1: Convert to grayscale

If the image is colored, convert it to grayscale so that intensity information can be processed directly.

Step 2: Enhance contrast

Apply **CLAHE** or histogram equalization.

CLAHE is generally preferred when different regions of the image have different contrast levels.

Step 3: Reduce noise

Apply a Gaussian filter with an appropriate kernel size.

Step 4: Detect edges

Apply **Canny, Sobel, or Prewitt**.

Step 5: Tune parameters

For Canny, select appropriate low and high thresholds. For Sobel or Prewitt, select suitable gradient thresholds.

The final pipeline is:

Low-Contrast Image

→ **Grayscale Conversion**

→ **CLAHE**

→ **Gaussian Filtering**

→ **Canny/Sobel Edge Detection**

→ **Improved Edge Map**

13. Analysis of the Solution

The preprocessing approach works because each stage addresses a different problem.

CLAHE improves local contrast and strengthens weak boundaries.

Gaussian filtering reduces unwanted high-frequency variations and prevents noise from being interpreted as edges.

Canny/Sobel then operates on an image in which meaningful intensity transitions are more prominent.

The combination is better than simply increasing contrast because excessive contrast enhancement alone can amplify noise. Similarly, excessive smoothing can remove genuine fine edges.

Therefore, preprocessing parameters must be selected carefully.

There is a balance:

Too little enhancement → weak edges remain undetected

Too much enhancement → noise and false edges increase

Too much smoothing → real fine edges disappear

A suitable combination provides the best result.

14. Expected Results

After preprocessing, the edge detection output should show:

- Clearer object boundaries.
- Stronger edges.
- Fewer missing edges.
- More continuous contours.
- Better separation between objects and background.
- Reduced influence of small noise variations.
- Improved input quality for later segmentation or object recognition.

For applications such as **medical imaging, industrial inspection, satellite images, and low-light photography**, this improvement can be especially important because weak boundaries may contain important information.

Conclusion

Low contrast makes edge detection difficult because the intensity difference between an object and its background is too small to produce strong gradients. Consequently, algorithms such as Sobel, Prewitt, and Canny may generate weak, broken, or missing edges.

Preprocessing solves this problem by **enhancing contrast and reducing noise before edge detection**. Histogram Equalization can improve global contrast, while **CLAHE is particularly effective for local contrast variations**. Gaussian smoothing can then reduce noise and prevent false edges. Applying Canny or Sobel after these preprocessing steps produces stronger and more reliable boundaries.

Thus, the most suitable approach can be summarized as:

Low Contrast → Contrast Enhancement (CLAHE) → Noise Reduction → Edge Detection → Clearer and More Reliable Edges

This approach provides a feasible solution because it directly addresses the reason the original edge detector failed while maintaining the important image structures.